

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

PRATHEETH S ANGARI (1BM23CS246)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “ Bio Inspired Systems (23CS5BSBIS)” carried out by **Pratheeth S Angari (1BM23CS246)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

Swathi Sridharan Assistant Professor Department of CSE, BMSCE	Dr Kavita Sooda Assistant Professor Department of CSE, BMSCE
---	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	28/08/2025	Genetic Algorithms for optimization	1-5
2	4/9/2025	Gene Expression Algorithm	6-9
3	11/9/2025	Particle Swarm Optimization(PSO)	10-13
4	9/10/2025	Ant Colony Optimization(ACO)	14-22
5	16/10/2025	Cuckoo Search	23-27
6	23/10/2025	Grey Wolf Optimization	28-33
7	30/10/2025	Parallel Cellular Algorithms	34-39

Github Link:

<https://github.com/pratheeth123/Bio-Inspired-Systems/tree/main>

Program 1

Genetic Algorithm for Optimization Problems

A Genetic Algorithm (GA) is an optimization technique inspired by the process of natural evolution. It works by creating a population of possible solutions and improving them over several generations using operations similar to biological evolution, such as selection, crossover, and mutation. In each generation, the best solutions are chosen based on a fitness function, and new solutions are created by modifying or combining them. Over time, the population evolves toward an optimal or near-optimal answer. GAs are useful for solving complex problems where traditional methods may fail or be too slow.

Algorithm:

Pseudocode:

```
Initialize population, size N, i = 0
fitness_score(population)
def selection(population)
    crossover(population)
    mutate(population)
while i < N
    print(Results)
    Initialize Results = fitness_score(population)
    print(Results)
```

The above algorithm tries to optimize the fitness of the population where chromosomes are represented as binary strings using mathematical functions/methods.

- Improve pseudocode
- Create work - image processing

Pseudocode/Algorithm.

Fitness function: Define a mathematical function to optimize

```
def fitness(population):
    return function(population)
```

Selection: selection:

Select a certain population whose criteria is met to pass onto next generation

```
def select(population):
    desired_value = 0.1
    if fit(population) < desired_value:
        remove
        population.remove()
    return population.
```

(crossover: combine parents to form a new child who is passed onto next generation)

```
def crossover(population):
    per-A = random.choices(population)
    per-B = random.choices(population)
    child = concatenate(per-A, per-B)
    to produce a child population append(child)
    return population.
```

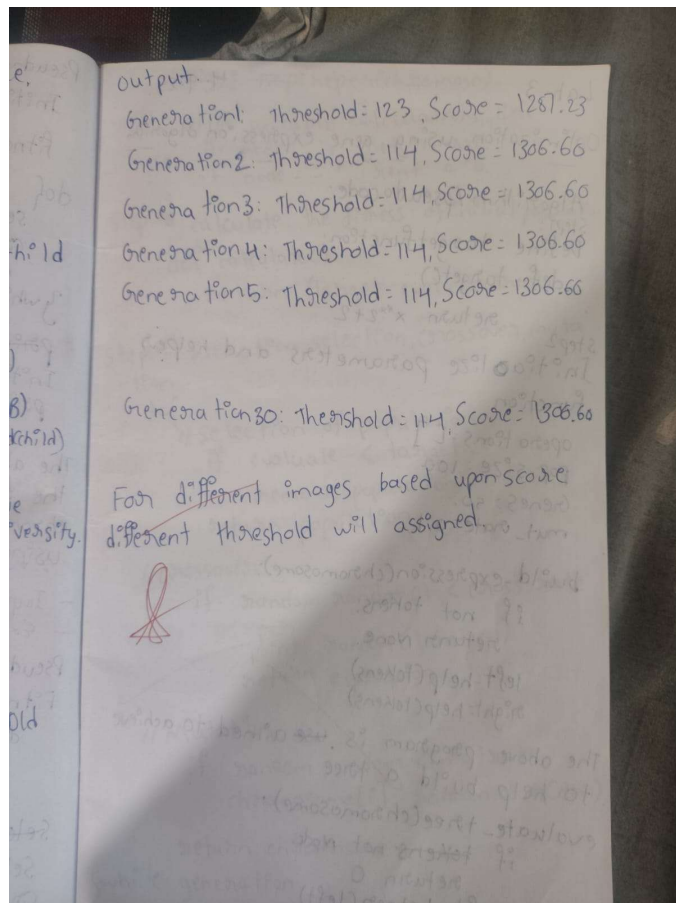
Mutation: mutate the population to preserve diversity

```
def mutation(population):
    population = flip(population)
    return population.
```

Objective:

The algorithm tries to find the threshold which distinguishes b/w foreground & background.

The range of threshold is (0-255)



```
import cv2
import numpy as np
import random
import urllib.request
import matplotlib.pyplot as plt
```

```
def url_to_image(url, flag=cv2.IMREAD_GRAYSCALE):
    req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
    with urllib.request.urlopen(req) as resp:
        arr = np.asarray(bytearray(resp.read()), dtype=np.uint8)
        img = cv2.imdecode(arr, flag)
    return img
url = "https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg"
img = url_to_image(url)
img = cv2.resize(img, (128, 128))
h, w = img.shape
```

```
def fitness(thresh):
    fg = img[img >= thresh]
    bg = img[img < thresh]
    if len(fg) == 0 or len(bg) == 0:
        return 0
    mean_fg = np.mean(fg)
```

```

mean_bg = np.mean(bg)
w_fg = len(fg)/img.size
w_bg = len(bg)/img.size
return w_fg*w_bg*(mean_fg - mean_bg)**2

POP_SIZE = 20
GENS = 30
MUT_RATE = 0.2

population = [random.randint(0, 255) for _ in range(POP_SIZE)]
best_thresh = None
best_score = 0

for gen in range(GENS):
    scores = [fitness(t) for t in population]
    idx = np.argmax(scores)
    if scores[idx] > best_score:
        best_score = scores[idx]
        best_thresh = population[idx]
    print(f"Generation {gen+1}: Best Threshold = {best_thresh}, Score = {best_score:.2f}")

    selected_idx = np.argsort(scores)[::-1][:POP_SIZE//2]
    selected = [population[i] for i in selected_idx]
    next_gen = []
    while len(next_gen) < POP_SIZE:
        parent = random.choice(selected)
        child = parent
        if random.random() < MUT_RATE:
            child = min(255, max(0, parent + random.randint(-10,10)))
        next_gen.append(child)
    population = next_gen

_, segmented = cv2.threshold(img, best_thresh, 255, cv2.THRESH_BINARY)
print("1BM23CS246")

plt.subplot(1,2,1)
plt.title("Original")
plt.imshow(img, cmap="gray")
plt.axis("off")
plt.subplot(1,2,2)
plt.title(f"Segmented (Thresh={best_thresh})")
plt.imshow(segmented, cmap="gray")
plt.axis("off")
plt.show()

```

Output :

```
Generation 21: Best Threshold = 14, Score = 5518.77
Generation 22: Best Threshold = 14, Score = 5518.77
Generation 23: Best Threshold = 14, Score = 5518.77
Generation 24: Best Threshold = 14, Score = 5518.77
Generation 25: Best Threshold = 14, Score = 5518.77
Generation 26: Best Threshold = 14, Score = 5518.77
Generation 27: Best Threshold = 14, Score = 5518.77
Generation 28: Best Threshold = 14, Score = 5518.77
Generation 29: Best Threshold = 14, Score = 5518.77
Generation 30: Best Threshold = 14, Score = 5518.77
1BM23CS246
```

Original



Segmented (Thresh=14)



Program 2:

Optimization Using Gene Expression Algorithm :

Gene expression is the biological process by which the information stored in a gene (DNA) is converted into a functional product, usually a protein. This process involves two main steps: transcription, where DNA is copied into mRNA, and translation, where mRNA is used to build proteins. These proteins then carry out various functions in the cell, determining an organism's traits, behavior, and responses. In simple terms, gene expression is how genetic information becomes an actual physical or functional result in a living organism.

Algorithm :

Lab-3
4-9-25
Optimization using gene expression algorithm

Algorithm/pseudocode:
Step 1:
Define target function:
def target():
 return $x^2 + 2$

Step 2:
Initialize parameters and helper function
operations: []
pop-size: 100
genes: 50
mut-rate: 0.1

build-expression(chromosome):
 if not tokens:
 return None
 left = help(tokens)
 right = help(tokens)

The above program is ~~use~~ aimed to achieve to help build a tree.

evaluate-tree(chromosome):
 if tokens not None:
 return 0
 left = help(left)

Step 3:
def

Step 4:
-tion.
def

while

9-25
right = ~~help~~ helper(chromosomes).
algorithm
// Do suitable calculations like
if node == '+': return a+b
if node == '*': return a*b

Step 3:
calculate the fitness of initial population
def calculate():
 return (target-evaluate-tree)

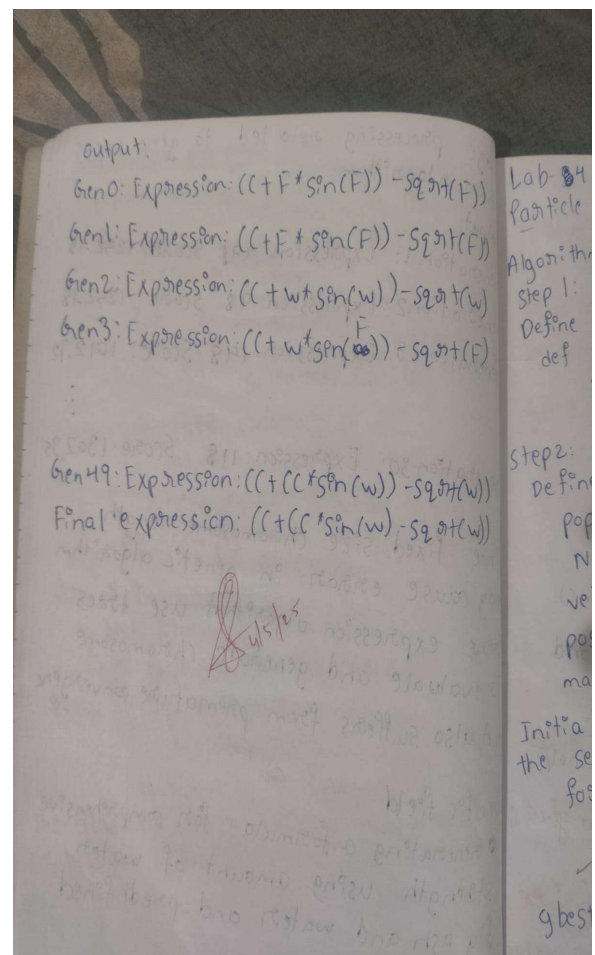
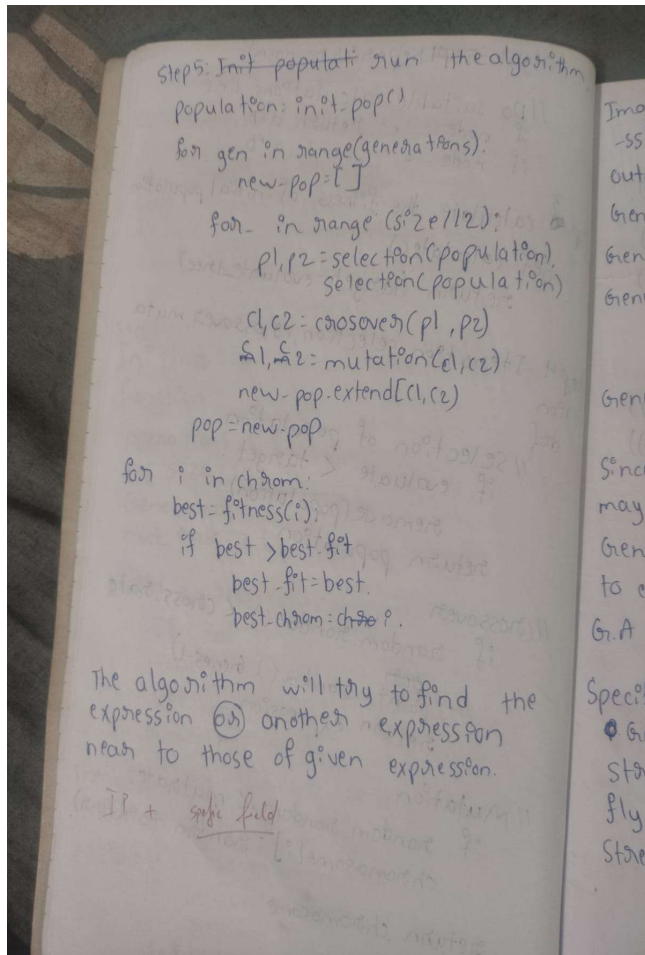
Step 4: Iteration, selection, crossover, mutation
-tion.
def
 // selection of population.
 if evaluate < target:
 remove(population)
 return population

 // crossover
 if random.random() < cross-rate
 point = random(1, genes-1)
 return expression

 // mutation.
 if random.random() < mut-rate:
 chromosome[i] = random.choice(Attn)

 return chromosome

while generation.



Code:

```

import cv2
import numpy as np
import random
import urllib.request
import matplotlib.pyplot as plt

def url_to_image(url, flag=cv2.IMREAD_GRAYSCALE):
    req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
    with urllib.request.urlopen(req) as resp:
        arr = np.asarray(bytearray(resp.read()), dtype=np.uint8)
        img = cv2.imdecode(arr, flag)
    return img

url = "https://i.imgur.com/Qr71crq.jpeg"

img = url_to_image(url)
img = cv2.resize(img, (128, 128))
h, w = img.shape

def fitness(thresh):
  
```

```

fg = img[img >= thresh]
bg = img[img < thresh]

if len(fg) == 0 or len(bg) == 0:
    return 0

mean_fg = np.mean(fg)
mean_bg = np.mean(bg)
w_fg = len(fg) / img.size
w_bg = len(bg) / img.size

return w_fg * w_bg * (mean_fg - mean_bg) ** 2

POP_SIZE = 20
ITER = 50
MUT_RATE = 0.25

population = [random.randint(0, 255) for _ in range(POP_SIZE)]
best_thresh = None
best_score = 0

for gen in range(1, ITER + 1):
    scores = [fitness(t) for t in population]
    idx = np.argmax(scores)

    if scores[idx] > best_score:
        best_score = scores[idx]
        best_thresh = population[idx]

    print(f'Gen {gen} : Threshold = {population[idx]} Score = {scores[idx]:.4f}')

    selected_idx = np.argsort(scores)[::-1][:POP_SIZE // 2]
    selected = [population[i] for i in selected_idx]

    next_population = []
    while len(next_population) < POP_SIZE:
        parent = random.choice(selected)
        child = parent

        if random.random() < MUT_RATE:
            child += random.randint(-12, 12)
            child = max(0, min(255, child))

        next_population.append(child)

    population = next_population

```

```
_ , segmented = cv2.threshold(img, best_thresh, 255, cv2.THRESH_BINARY)
print("1BM23CS246")
```

```
plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
plt.title("Image")
plt.imshow(img, cmap="gray")
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
plt.title(f'Segmented (Thresh={best_thresh})')
plt.imshow(segmented, cmap="gray")
plt.axis("off")
```

```
plt.show()
```

Output :

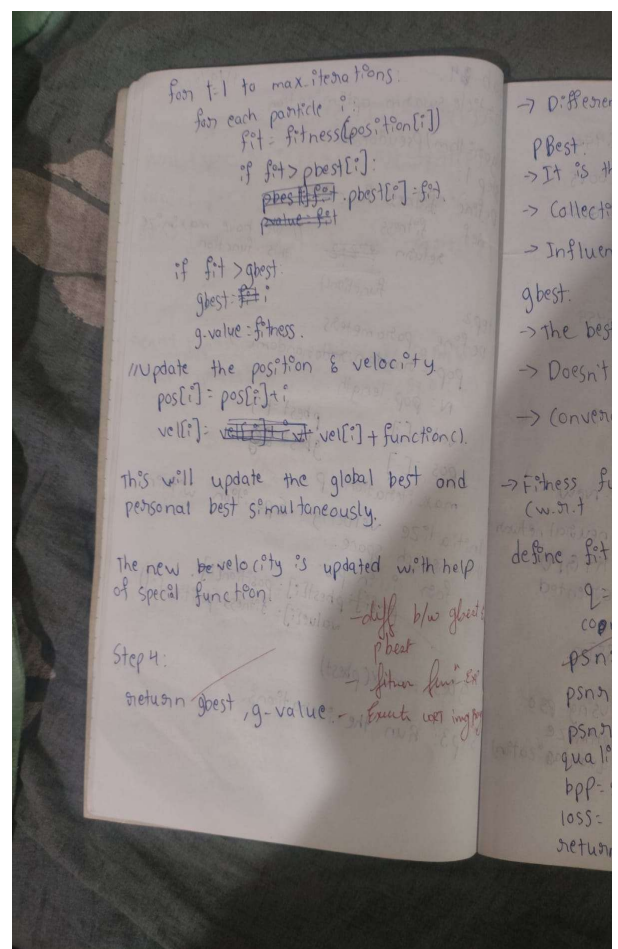
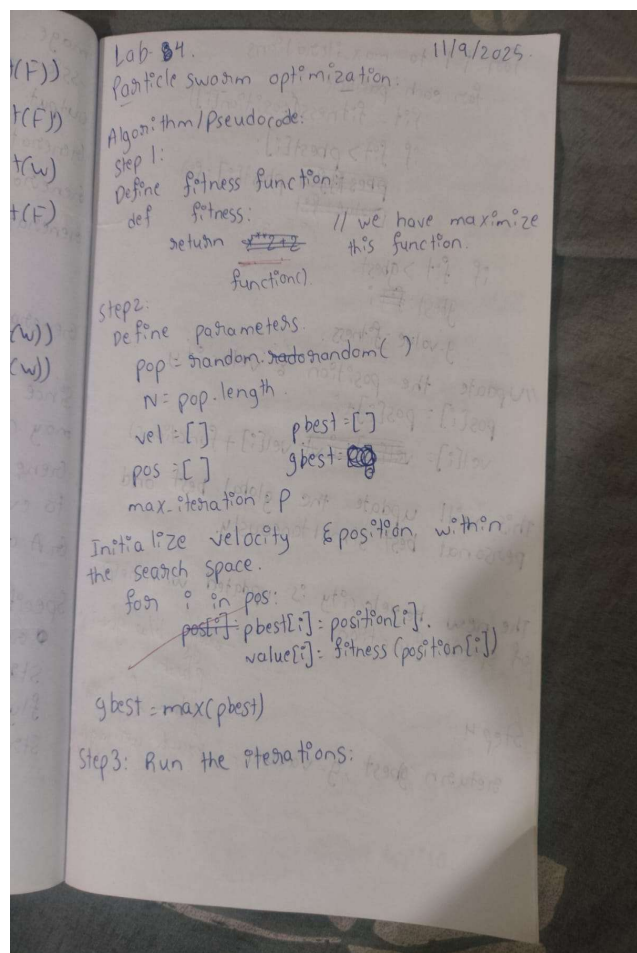


Program 3:

Particle Swarm Optimization :

Particle Swarm Optimization (PSO) is a population-based optimization technique inspired by the collective behavior of birds flocking or fish schooling. In PSO, each solution, called a particle, moves through the search space by adjusting its position based on its own best-known value and the best-known value found by the entire swarm. Through continuous updating of velocity and position, the particles naturally converge toward optimal or near-optimal solutions. PSO is widely used because it is simple, efficient, and works well for complex optimization problems where traditional methods struggle.

Algorithm :



→ Difference b/w pbest and gbest
 pBest:
 → It is the best position of a particle so far.
 → Collectively contributes towards gbest.
 → Influencing the particle to return a good position.
 gbest:
 → The best position found by any particle.
 → Doesn't contribute towards pbest.
 → Converges to the best-known solution.
 → Fitness function expression:
 (w.r.t to image compression).
 define fitness(q):
 $q = \max(q, 1e-6)$
 compressed = quantize(img, q).
 $psnr_r = psnr(img[...], 0], compressed[...], 0]$
 $psnr_g = psnr(img[...], 1], compressed[...], 1]$
 $psnr_b = psnr(img[...], 2], compressed[...], 2]$
 quality = (psnr_r + psnr_g + psnr_b) / 3
 bpp = 8/q.
 loss = (100 - quality) + bpp * 10.
 return loss.

Output:
 Iteration 1, q: 0.8000 loss: 260.6455
 Iteration 2, q: 0.6500 loss: 260.0015
 Iteration 3, q: 0.5000 loss: 242.6455
 Iteration 4, q: 0.5000 loss: 242.6455
 Results → Image compression
 old } - diffn - phy² ↑ cost
 latent }
 Image compression old new
 spatial compress neural net
 - fractal K+GPU
 blocks oriented
 metrics: MSE, PSNR SSIM
 How PSO is used: using PSO to find blocks in the given image
 (using PSO to optimize quantization)

Code :

```

import numpy as np
from skimage.metrics import peak_signal_noise_ratio as psnr
from skimage import data
import random
import matplotlib.pyplot as plt

img = data.chelsea().astype(np.float32) / 255.0

def quantize(image, q):
    q = max(q, 1e-6)
    return np.round(image / q) * q

def objective(q):
    q = max(1e-6, q)
    compressed = quantize(img, q)

    psnr_r = psnr(img[...], 0], compressed[...], 0])
    psnr_g = psnr(img[...], 1], compressed[...], 1])
  
```

```

    psnr_b = psnr(img[..., 2], compressed[..., 2])
    quality = (psnr_r + psnr_g + psnr_b) / 3.0
    bpp = 8 / q
    loss = (100 - quality) + bpp * 10
    return loss

num_particles = 30
num_iterations = 50
w = 0.5
c1 = 1.5
c2 = 1.5

particles = np.random.uniform(1e-6, 0.5, size=num_particles)
velocities = np.zeros(num_particles)

pbest = particles.copy()
pbest_scores = np.array([objective(p) for p in particles])
gbest_idx = np.argmin(pbest_scores)
gbest = pbest[gbest_idx]
gbest_score = pbest_scores[gbest_idx]

for iter in range(num_iterations):
    for i in range(num_particles):
        r1, r2 = random.random(), random.random()
        velocities[i] = (w * velocities[i] +
                        c1 * r1 * (pbest[i] - particles[i]) +
                        c2 * r2 * (gbest - particles[i]))
        particles[i] += velocities[i]

        particles[i] = np.clip(particles[i], 1e-6, 0.5)

        score = objective(particles[i])

        if score < pbest_scores[i]:
            pbest[i] = particles[i]
            pbest_scores[i] = score

    gbest_idx = np.argmin(pbest_scores)
    if pbest_scores[gbest_idx] < gbest_score:
        gbest = pbest[gbest_idx]
        gbest_score = pbest_scores[gbest_idx]

    print(f'Iteration {iter+1}, Best q = {gbest:.6f}, Loss = {gbest_score:.4f}')

print(f'\nOptimal quantization step size q: {gbest}')
print("IBM23CS246")

compressed_img = quantize(img, gbest)

```



```
plt.subplot(1, 2, 1)
plt.title('Original Image')
plt.imshow(img)
plt.axis('off')

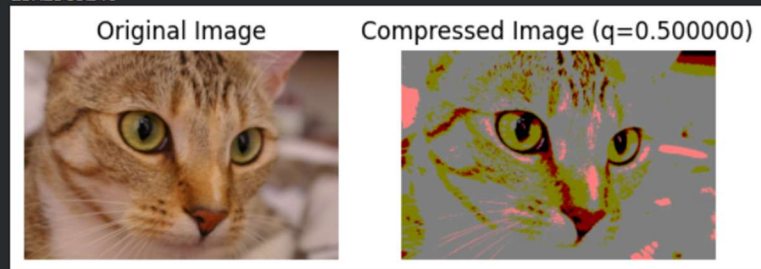
plt.subplot(1, 2, 2)
plt.title(f'Compressed Image (q={gbest:.6f})')
plt.imshow(compressed_img)
plt.axis('off')

plt.show()
```

Output :

```
Iteration 44, Best q = 0.500000, Loss = 242.6455
Iteration 45, Best q = 0.500000, Loss = 242.6455
Iteration 46, Best q = 0.500000, Loss = 242.6455
Iteration 47, Best q = 0.500000, Loss = 242.6455
Iteration 48, Best q = 0.500000, Loss = 242.6455
Iteration 49, Best q = 0.500000, Loss = 242.6455
Iteration 50, Best q = 0.500000, Loss = 242.6455
```

```
Optimal quantization step size q: 0.5
1BM23CS246
```

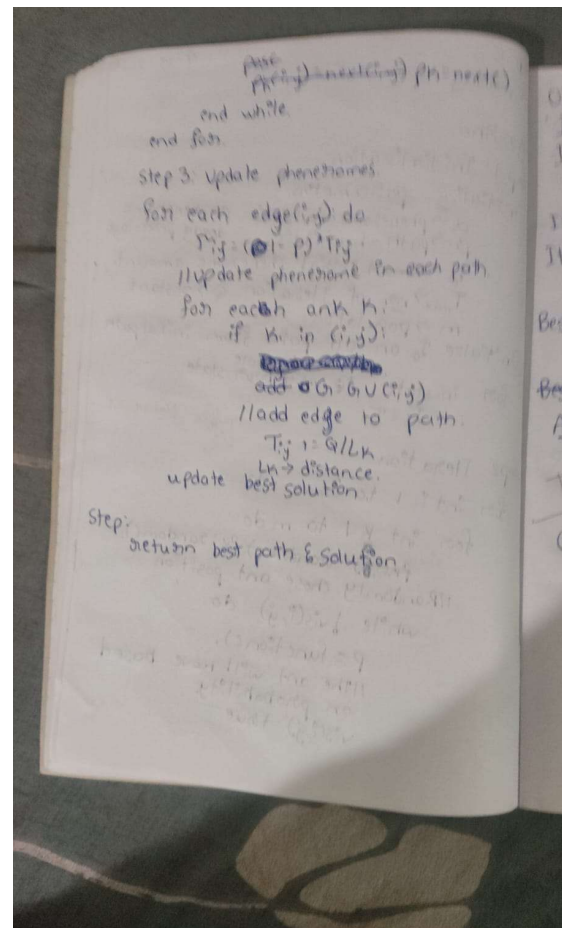
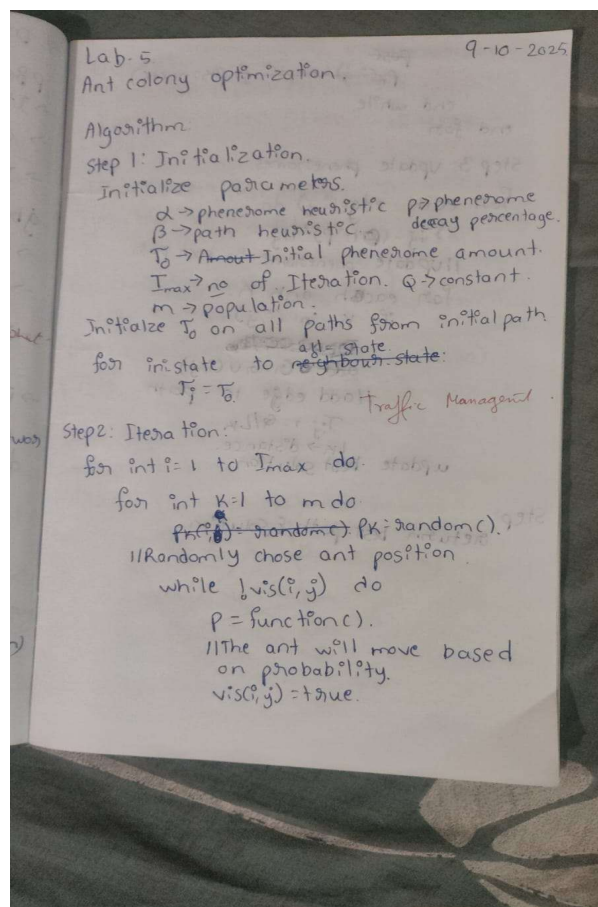


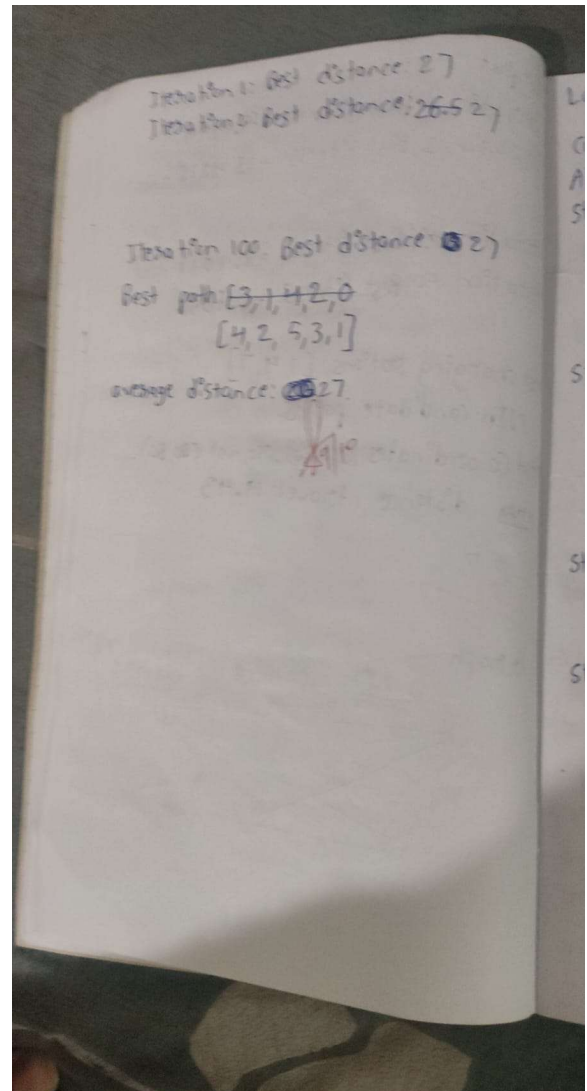
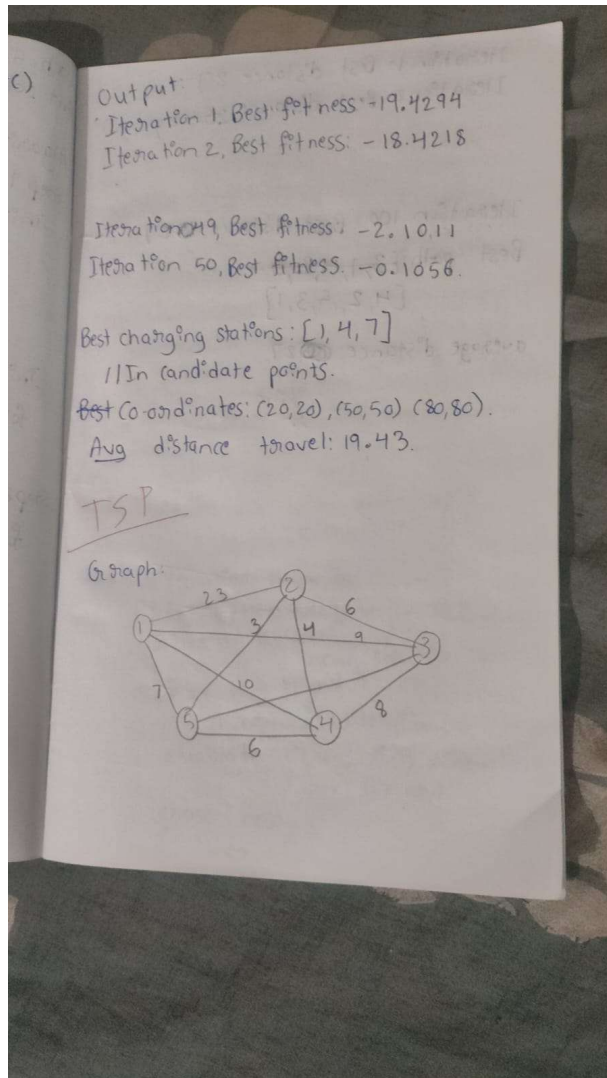
Program 4:

Ant Colony Optimization :

Ant Colony Optimization (ACO) is a nature-inspired optimization technique based on the foraging behavior of real ants. When ants search for food, they deposit pheromone trails along paths, and over time, shorter or better paths accumulate stronger pheromone levels because more ants follow them. ACO simulates this process by having artificial ants explore possible solutions and reinforce good ones with virtual pheromones, allowing the algorithm to converge toward the optimal path or solution. It is especially effective for solving complex combinatorial problems like routing, scheduling, and network optimization.

Algorithm :





Code for Traffic Management :

```
import numpy as np
```

```
class ACOChargingStations:
```

```
    def __init__(self, candidate_stations, demand_points, n_stations_to_select,
                  n_ants=20, n_iterations=50, alpha=1, beta=2, rho=0.1):
```

```
        """
```

```
        candidate_stations: list of (x,y) tuples for potential station locations
```

```
        demand_points: list of (x,y) tuples for EV user demand locations
```

```
        n_stations_to_select: how many stations to choose
```

```
        """
```

```
        self.candidates = np.array(candidate_stations)
```

```
        self.demands = np.array(demand_points)
```

```
        self.n_select = n_stations_to_select
```

```
        self.n_ants = n_ants
```

```
        self.n_iterations = n_iterations
```

```

self.alpha = alpha
self.beta = beta
self.rho = rho

self.n_candidates = len(candidate_stations)
self.pheromone = np.ones(self.n_candidates)
self.heuristic = self.compute_heuristic()

def compute_heuristic(self):
    heur = []
    for station in self.candidates:
        dists = np.linalg.norm(self.demands - station, axis=1)
        avg_dist = np.mean(dists)
        heur.append(1.0 / (avg_dist + 1e-6))
    return np.array(heur)

def construct_solution(self):
    solution = []
    pheromone = self.pheromone ** self.alpha
    heuristic = self.heuristic ** self.beta
    probs = pheromone * heuristic
    probs = probs / probs.sum()

    selected = set()
    candidates_left = set(range(self.n_candidates))
    while len(selected) < self.n_select:
        chosen = np.random.choice(list(candidates_left),
p=probs[list(candidates_left)]/probs[list(candidates_left)].sum())
        selected.add(chosen)
        candidates_left.remove(chosen)
    return list(selected)

def evaluate_solution(self, solution):

    selected_stations = self.candidates[solution]
    total_dist = 0
    for demand in self.demands:
        dists = np.linalg.norm(selected_stations - demand, axis=1)
        total_dist += np.min(dists)
    avg_dist = total_dist / len(self.demands)

    return -avg_dist

def update_pheromone(self, all_solutions, all_scores):
    self.pheromone *= (1 - self.rho) # evaporation
    for sol, score in zip(all_solutions, all_scores):
        for station_idx in sol:
            self.pheromone[station_idx] += max(0, score) # deposit pheromone

```

```

def run(self):
    best_solution = None
    best_score = -float('inf')

    for iteration in range(self.n_iterations):
        all_solutions = []
        all_scores = []

        for _ in range(self.n_ants):
            sol = self.construct_solution()
            score = self.evaluate_solution(sol)
            all_solutions.append(sol)
            all_scores.append(score)

            if score > best_score:
                best_score = score
                best_solution = sol

        self.update_pheromone(all_solutions, all_scores)
        print(f"Iteration {iteration+1}, Best Fitness (neg avg dist): {best_score:.4f}")

    return best_solution, -best_score

candidate_stations = [
    (10,10), (20,20), (30,10), (40,40), (50,50),
    (60,60), (70,10), (80,80), (90,90), (25,75)
]

demand_points = [
    (12,15), (22,25), (28,12), (41,39), (48,52),
    (58,62), (69,9), (77,82), (88,91), (24,70),
    (15,85), (35,60), (55,45), (65,30), (75,20)
]

n_stations_to_select = 3

aco = ACOChargingStations(candidate_stations, demand_points, n_stations_to_select)
best_stations, best_avg_distance = aco.run()

print("\nBest charging station locations (indices):", best_stations)
print("Best charging station coordinates:")
for idx in best_stations:
    print(f" {candidate_stations[idx]}")
print(f"Average distance EVs travel to nearest station: {best_avg_distance:.2f}")

```

Code for TSP :

```
import numpy as np
```

```
class ACO_TSP:
```

```
    def __init__(self, dist_matrix, n_ants, n_iterations, decay, alpha=1, beta=2):
```

```
        """
```

```
        dist_matrix: 2D numpy array of distances between cities
```

```
        n_ants: number of ants per iteration
```

```
        n_iterations: number of iterations to run
```

```
        decay: pheromone evaporation coefficient ( $0 < \text{decay} < 1$ )
```

```
        alpha: pheromone importance
```

```
        beta: heuristic importance (usually inverse distance)
```

```
        """
```

```
        self.dist_matrix = dist_matrix
```

```
        self.n_cities = dist_matrix.shape[0]
```

```
        self.n_ants = n_ants
```

```
        self.n_iterations = n_iterations
```

```
        self.decay = decay
```

```
        self.alpha = alpha
```

```
        self.beta = beta
```

```
        self.pheromone = np.ones((self.n_cities, self.n_cities)) # initial pheromone
```

```
    def run(self):
```

```
        best_distance = float('inf')
```

```
        best_path = None
```

```
        for iteration in range(self.n_iterations):
```

```
            all_paths = self.construct_solutions()
```

```
            self.evaporate_pheromone()
```

```
            self.deposit_pheromone(all_paths)
```

```
            distances = [self.path_length(path) for path in all_paths]
```

```
            avg_distance = sum(distances) / len(distances)
```

```
            min_dist_idx = np.argmin(distances)
```

```
            if distances[min_dist_idx] < best_distance:
```

```
                best_distance = distances[min_dist_idx]
```

```
                best_path = all_paths[min_dist_idx]
```

```
        print(f"Iteration {iteration+1}: Best distance = {best_distance}, Average distance = {avg_distance}")
```

```
        return best_path, best_distance
```

```
    def construct_solutions(self):
```

```
        paths = []
```

```
        for _ in range(self.n_ants):
```

```
            path = self.construct_solution()
```

```

        paths.append(path)
    return paths

def construct_solution(self):
    path = []
    visited = set()

    current_city = np.random.randint(self.n_cities)
    path.append(current_city)
    visited.add(current_city)

    while len(path) < self.n_cities:
        next_city = self.select_next_city(current_city, visited)
        path.append(next_city)
        visited.add(next_city)
        current_city = next_city

    return path

def select_next_city(self, current_city, visited):
    pheromone = self.pheromone[current_city]
    distances = self.dist_matrix[current_city]

    prob = []
    for city in range(self.n_cities):
        if city not in visited:
            tau = pheromone[city] ** self.alpha
            eta = (1.0 / distances[city]) ** self.beta if distances[city] > 0 else 0
            prob.append(tau * eta)
        else:
            prob.append(0)

    prob = np.array(prob)
    prob_sum = prob.sum()
    if prob_sum == 0:
        choices = [city for city in range(self.n_cities) if city not in visited]
        return np.random.choice(choices)
    prob = prob / prob_sum

    return np.random.choice(range(self.n_cities), p=prob)

def path_length(self, path):
    length = 0
    for i in range(len(path)):
        length += self.dist_matrix[path[i - 1], path[i]]
    return length

def evaporate_pheromone(self):

```

```

self.pheromone *= (1 - self.decay)

def deposit_pheromone(self, paths):
    for path in paths:
        length = self.path_length(path)
        for i in range(len(path)):
            from_city = path[i - 1]
            to_city = path[i]
            self.pheromone[from_city, to_city] += 1.0 / length
            self.pheromone[to_city, from_city] += 1.0 / length

if __name__ == "__main__":
    # Example distance matrix for 5 cities (symmetric)
    dist_matrix = np.array([
        [0, 23, 9, 10, 7],
        [23, 0, 6, 4, 3],
        [9, 6, 0, 8, 1],
        [10, 4, 8, 0, 6],
        [7, 3, 1, 6, 0]
    ])

    aco = ACO_TSP(dist_matrix, n_ants=10, n_iterations=100, decay=0.1, alpha=1, beta=2)
    best_path, best_distance = aco.run()

    print("Best path:", best_path)
    print("Best distance:", best_distance)

```

Output :

```

Iteration 43, Best Fitness (neg avg dist): -19.4294
Iteration 44, Best Fitness (neg avg dist): -19.4294
Iteration 45, Best Fitness (neg avg dist): -19.4294
Iteration 46, Best Fitness (neg avg dist): -19.4294
Iteration 47, Best Fitness (neg avg dist): -19.4294
Iteration 48, Best Fitness (neg avg dist): -19.4294
Iteration 49, Best Fitness (neg avg dist): -19.4294
Iteration 50, Best Fitness (neg avg dist): -19.4294
1BM23CS246

Best charging station locations (indices): [np.int64(1), np.int64(4), np.int64(7)]
Best charging station coordinates:
(20, 20)
(50, 50)
(80, 80)
Average distance EVs travel to nearest station: 19.43

```



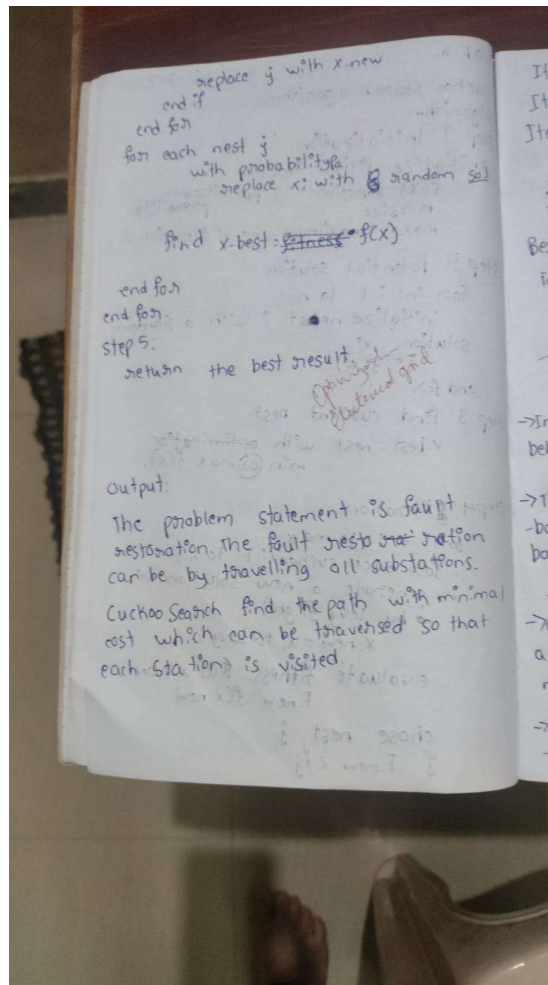
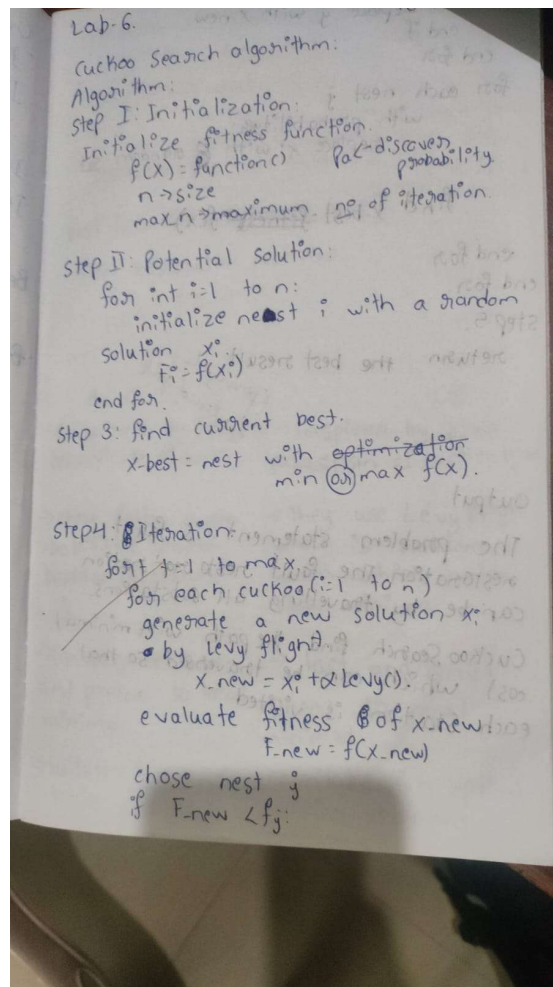
```
Iteration 92: Best distance = 27, Average distance = 27.1
Iteration 93: Best distance = 27, Average distance = 27.1
Iteration 94: Best distance = 27, Average distance = 27.2
Iteration 95: Best distance = 27, Average distance = 27.2
Iteration 96: Best distance = 27, Average distance = 27.2
Iteration 97: Best distance = 27, Average distance = 27.2
Iteration 98: Best distance = 27, Average distance = 27.0
Iteration 99: Best distance = 27, Average distance = 27.4
Iteration 100: Best distance = 27, Average distance = 27.1
1BM23CS246
Best path: [2, np.int64(4), np.int64(1), np.int64(3), np.int64(0)]
Best distance: 27
```

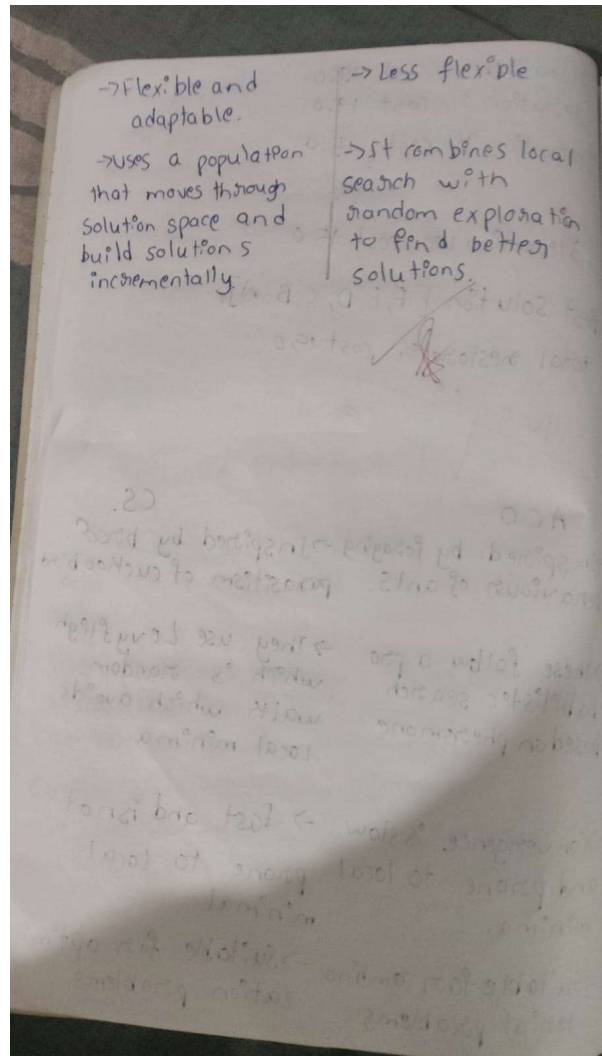
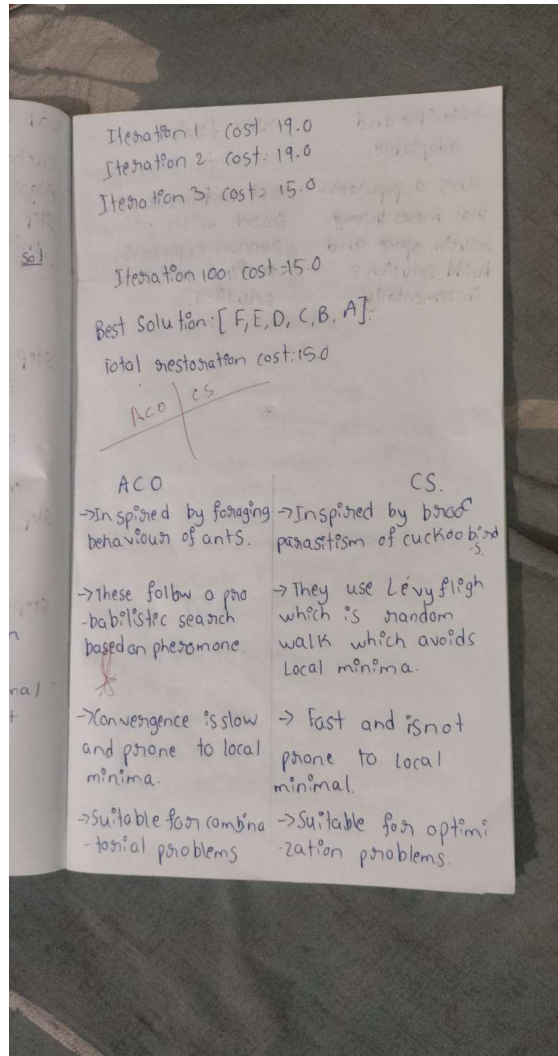
Program 5 :

Cuckoo Search

The Cuckoo Search (CS) algorithm is a nature-inspired optimization technique based on the brood parasitism behavior of certain cuckoo species, which lay their eggs in the nests of other birds. In CS, each cuckoo represents a potential solution, and new solutions are generated using Levy flights, allowing large random steps that help escape local optima. Host birds may discover alien eggs with some probability, leading to the replacement of poor solutions. Through this balance of exploration and exploitation, Cuckoo Search efficiently searches for the global optimum in complex optimization problems.

Algorithm :





Code:

```
import random
import numpy as np
```

```
class Grid:
    def __init__(self, nodes, edges):
        self.nodes = nodes
```

```

self.edges = edges
self.adjacency_matrix = self.create_adjacency_matrix()

def create_adjacency_matrix(self):
    matrix = np.zeros((len(self.nodes), len(self.nodes)))
    for edge in self.edges:
        u, v, weight = edge
        matrix[u][v] = matrix[v][u] = weight
    return matrix

def get_neighbors(self, node):
    return [i for i in range(len(self.nodes)) if self.adjacency_matrix[node][i] > 0]

class CuckooSearch:
    def __init__(self, grid, n_cuckoos=15, max_iter=100, pa=0.3):
        self.grid = grid
        self.n_cuckoos = n_cuckoos
        self.max_iter = max_iter
        self.pa = pa
        self.best_solution = None
        self.best_cost = float('inf')
        self.solutions = [self.random_solution() for _ in range(n_cuckoos)]

    def random_solution(self):
        start_node = random.randint(0, len(self.grid.nodes) - 1)
        path = [start_node]
        visited = {start_node}
        while len(path) < len(self.grid.nodes):
            neighbors = [n for n in self.grid.get_neighbors(path[-1]) if n not in visited]
            if not neighbors:
                break
            next_node = random.choice(neighbors)
            path.append(next_node)
            visited.add(next_node)
        return path

    def fitness(self, solution):
        total_cost = 0
        for i in range(len(solution) - 1):
            u, v = solution[i], solution[i + 1]
            if self.grid.adjacency_matrix[u][v] > 0:
                total_cost += self.grid.adjacency_matrix[u][v]
            else:
                total_cost += 1e6 # penalty for invalid edge
        if len(solution) < len(self.grid.nodes):
            total_cost += 1e5 # penalty for incomplete path
        return total_cost

    def levy_flight(self, current_solution):

```

```

new_solution = current_solution.copy()
if len(new_solution) > 1:
    idx1, idx2 = random.sample(range(len(new_solution)), 2)
    new_solution[idx1], new_solution[idx2] = new_solution[idx2], new_solution[idx1]
return new_solution

def update(self):
    for i in range(self.n_cuckoos):
        new_solution = self.levy_flight(self.solutions[i])
        new_cost = self.fitness(new_solution)
        if new_cost < self.best_cost:
            self.best_solution = new_solution
            self.best_cost = new_cost
        if random.random() < self.pa:
            self.solutions[i] = self.random_solution()
        else:
            if new_cost < self.fitness(self.solutions[i]):
                self.solutions[i] = new_solution

def run(self):
    for iteration in range(self.max_iter):
        self.update()
        print(f"Iteration {iteration + 1}: Best Cost = {self.best_cost}")
    return self.best_solution, self.best_cost

nodes = ['A', 'B', 'C', 'D', 'E', 'F']
edges = [
    (0, 1, 4), # A-B
    (0, 2, 3), # A-C
    (1, 2, 1), # B-C
    (1, 3, 7), # B-D
    (2, 3, 5), # C-D
    (2, 4, 8), # C-E
    (3, 4, 2), # D-E
    (3, 5, 6), # D-F
    (4, 5, 3) # E-F
]

grid = Grid(nodes, edges)
cuckoo_search = CuckooSearch(grid, n_cuckoos=20, max_iter=50, pa=0.3)

best_solution, best_cost = cuckoo_search.run()
print("1BM23CS246")

print("\nBest solution path:")
for idx in best_solution:
    print(nodes[idx])
print(f"\nTotal restoration cost: {best_cost}")

```

Output :

```
Iteration 41: Best Cost = 15.0  
Iteration 42: Best Cost = 15.0  
Iteration 43: Best Cost = 15.0  
Iteration 44: Best Cost = 15.0  
Iteration 45: Best Cost = 15.0  
Iteration 46: Best Cost = 15.0  
Iteration 47: Best Cost = 15.0  
Iteration 48: Best Cost = 15.0  
Iteration 49: Best Cost = 15.0  
Iteration 50: Best Cost = 15.0  
1BM23CS246
```

Best solution path:

A
B
C
D
E
F

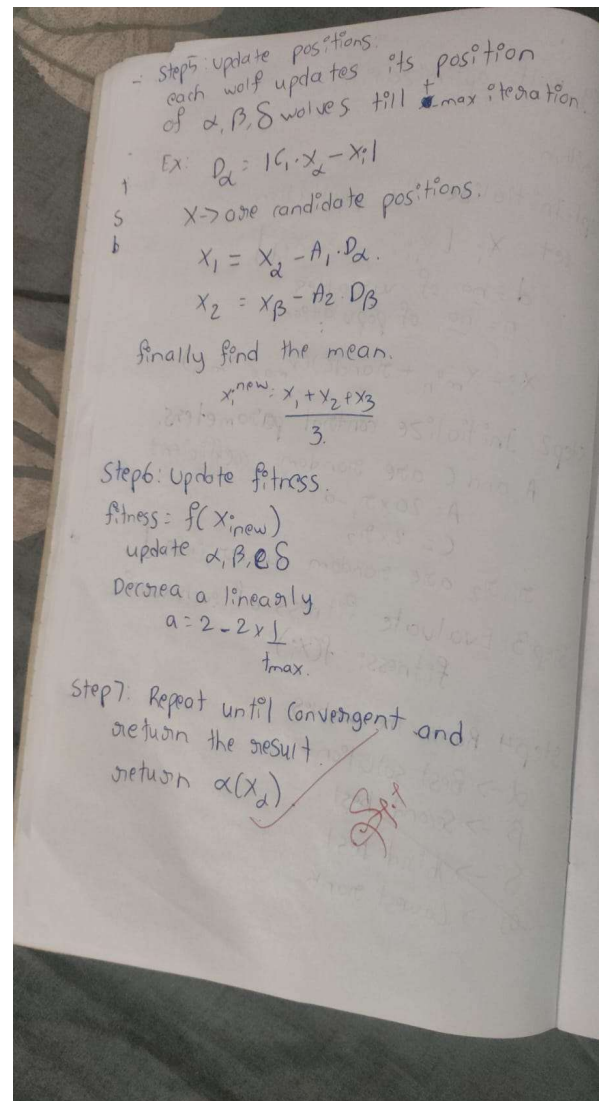
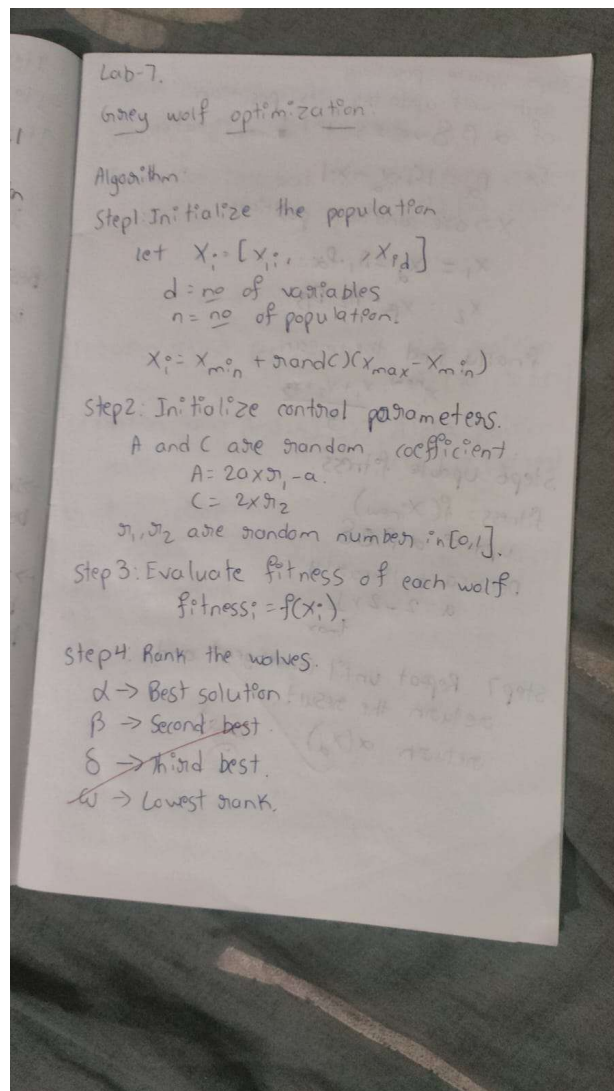
Total restoration cost: 15.0

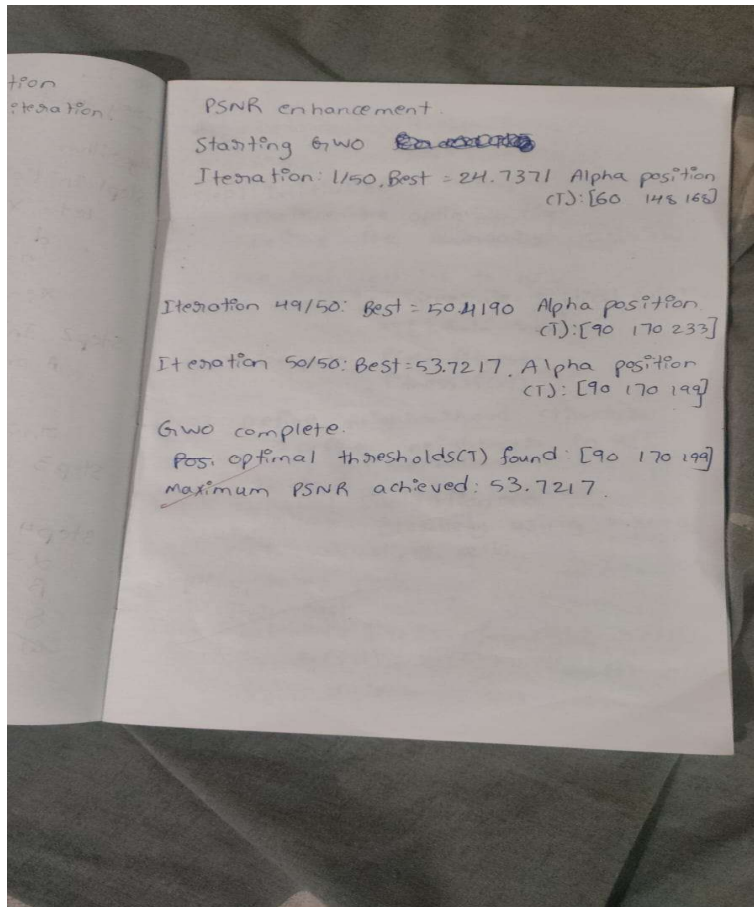
Program 6:

Grey Wolf Optimization :

Grey Wolf Optimization (GWO) is a nature-inspired optimization algorithm based on the hunting behavior and social hierarchy of grey wolves. In GWO, the best three solutions in the population are treated as alpha, beta, and delta wolves, which guide the rest of the search agents (omega wolves). The wolves encircle prey (the optimal solution) by updating their positions based on the leaders, gradually narrowing in on the best region of the search space. Through this cooperative hunting mechanism, GWO efficiently balances exploration and exploitation, making it effective for solving complex optimization problems.

Algorithm :





Code:

```
import numpy as np
import cv2
from skimage.metrics import peak_signal_noise_ratio as calculate_psnr
import random
import matplotlib.pyplot as plt

def objective_function(I_original, thresholds):
    I_enhanced = multi_level_he(I_original, thresholds)

    try:
        psnr_value = calculate_psnr(I_original, I_enhanced, data_range=255)
    except Exception:
        return 1000.0

    return -psnr_value

def multi_level_he(image, thresholds):
    T = sorted(list(thresholds))

    segments = [0] + T + [255]
```

```

enhanced_image = np.zeros_like(image, dtype=np.uint8)

for i in range(len(segments) - 1):
    t_start = segments[i]
    t_end = segments[i+1]

    if i == 0:
        mask = (image >= t_start) & (image <= t_end)
    else:
        mask = (image > t_start) & (image <= t_end)

    segment_pixels = image[mask]

    if len(segment_pixels) > 0:
        min_val = segment_pixels.min()
        max_val = segment_pixels.max()

        if min_val == max_val:
            enhanced_segment = segment_pixels
        else:
            new_min = t_start
            new_max = t_end

            enhanced_segment = ((segment_pixels.astype(np.float32) - min_val) * (new_max -
new_min) / (max_val - min_val)) + new_min
            enhanced_segment = np.clip(enhanced_segment, new_min, new_max).astype(np.uint8)

        enhanced_image[mask] = enhanced_segment

return enhanced_image

def grey_wolf_optimizer_mlhe(image, n_thresholds=3, num_wolves=15, max_iter=50):
    print(f'Starting GWO for MLHE (Wolves: {num_wolves}, Iterations: {max_iter})...')

    dim = n_thresholds
    lb = np.ones(dim) * 1
    ub = np.ones(dim) * 254

    positions = np.random.uniform(lb, ub, (num_wolves, dim))

    Alpha_pos = np.zeros(dim)
    Alpha_score = float('inf')
    Beta_pos = np.zeros(dim)
    Beta_score = float('inf')
    Delta_pos = np.zeros(dim)
    Delta_score = float('inf')

    psnr_history = []

```

```

for l in range(max_iter):

    for i in range(num_wolves):
        current_thresholds = np.sort(positions[i, :])
        fitness = objective_function(image, current_thresholds)

        if fitness < Alpha_score:
            Delta_score, Delta_pos = Beta_score, Beta_pos.copy()
            Beta_score, Beta_pos = Alpha_score, Alpha_pos.copy()
            Alpha_score, Alpha_pos = fitness, current_thresholds.copy()
        elif fitness < Beta_score:
            Delta_score, Delta_pos = Beta_score, Beta_pos.copy()
            Beta_score, Beta_pos = fitness, current_thresholds.copy()
        elif fitness < Delta_score:
            Delta_score, Delta_pos = fitness, current_thresholds.copy()

    psnr_history.append(-Alpha_score)

a = 2 - 1 * (2 / max_iter)

for i in range(num_wolves):
    for j in range(dim):
        r1, r2 = random.random(), random.random()
        A1 = 2 * a * r1 - a
        C1 = 2 * r2
        D_alpha = abs(C1 * Alpha_pos[j] - positions[i, j])

        r1, r2 = random.random(), random.random()
        A2 = 2 * a * r1 - a
        C2 = 2 * r2
        D_beta = abs(C2 * Beta_pos[j] - positions[i, j])

        r1, r2 = random.random(), random.random()
        A3 = 2 * a * r1 - a
        C3 = 2 * r2
        D_delta = abs(C3 * Delta_pos[j] - positions[i, j])

        X1 = Alpha_pos[j] - A1 * D_alpha
        X2 = Beta_pos[j] - A2 * D_beta
        X3 = Delta_pos[j] - A3 * D_delta

        positions[i, j] = (X1 + X2 + X3) / 3

    positions[i, j] = np.clip(positions[i, j], lb[j], ub[j])

    print(f"Iteration {l+1}/{max_iter}: Best PSNR = {-Alpha_score:.4f} dB, Alpha Position (T): {Alpha_pos.astype(int)}")

print("\nGWO Complete.")

```

```

print(f'Optimal Thresholds (T) found: {Alpha_pos.astype(int)}')
print(f'Maximum PSNR achieved: {-Alpha_score:.4f} dB')

best_enhanced_image = multi_level_he(image, Alpha_pos)
return best_enhanced_image, image, psnr_history

try:
    img_path = 'low_contrast_image.jpg'
    original_img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    if original_img is None:
        print("Image not found. Creating a synthetic low-contrast image.")
        original_img = (np.random.rand(150, 150) * 80 + 90).astype(np.uint8)
except:
    print("Error loading image. Creating a synthetic low-contrast image.")
    original_img = (np.random.rand(150, 150) * 80 + 90).astype(np.uint8)

enhanced_img, original_img, history = grey_wolf_optimizer_mlhe(
    original_img,
    n_thresholds=3,
    num_wolves=15,
    max_iter=50
)

print("1BM23CS246")
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(original_img, cmap='gray')
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].imshow(enhanced_img, cmap='gray')
axes[1].set_title(f'GWO Enhanced (Final PSNR: {history[-1]:.2f} dB)')
axes[1].axis('off')

axes[2].plot(history)
axes[2].set_title('PSNR Convergence Over Iterations')
axes[2].set_xlabel('Iteration')
axes[2].set_ylabel('PSNR (dB)')
axes[2].grid(True)

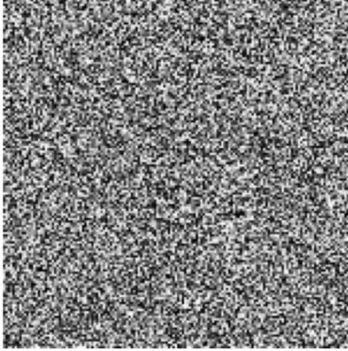
plt.tight_layout()
plt.show()

```

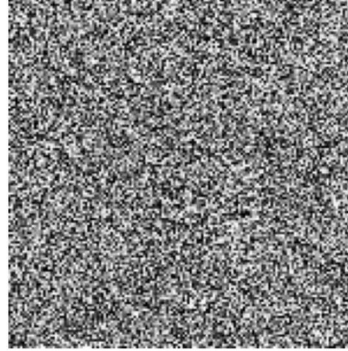
Output :

GWO Complete.
Optimal Thresholds (T) found: [2 90 169]
Maximum PSNR achieved: 52.1915 dB
1BM23CS246

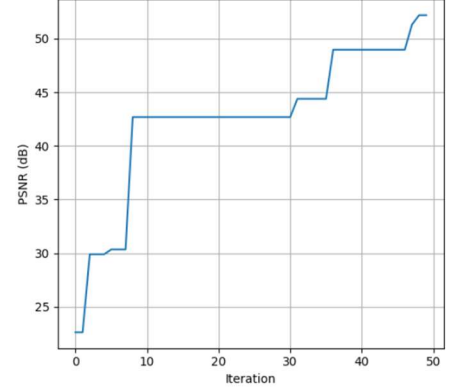
Original Image



GWO Enhanced (Final PSNR: 52.19 dB)



PSNR Convergence Over Iterations

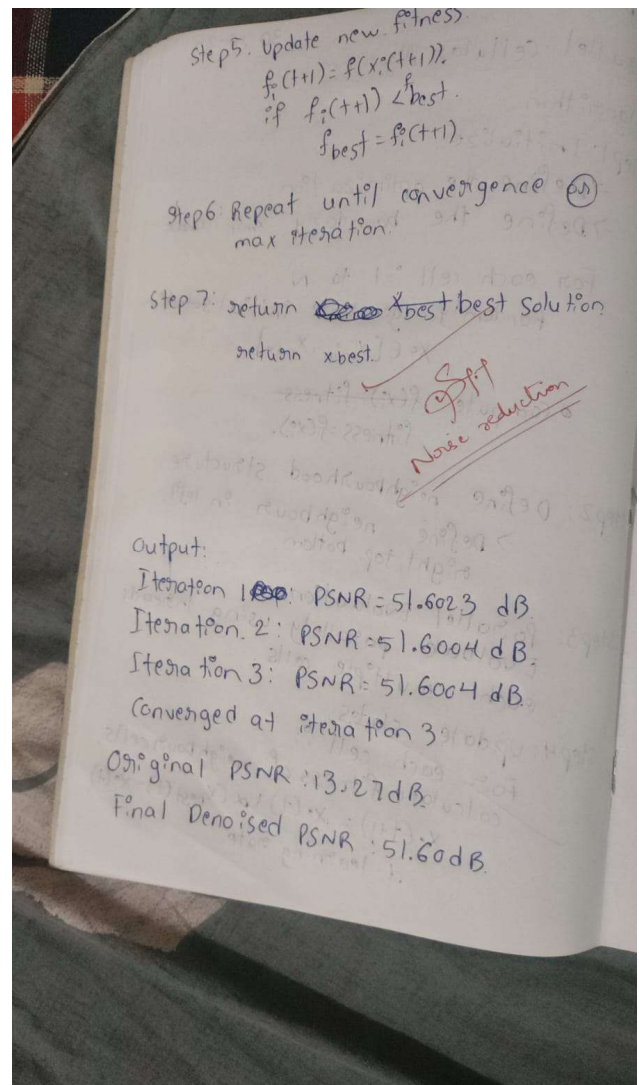
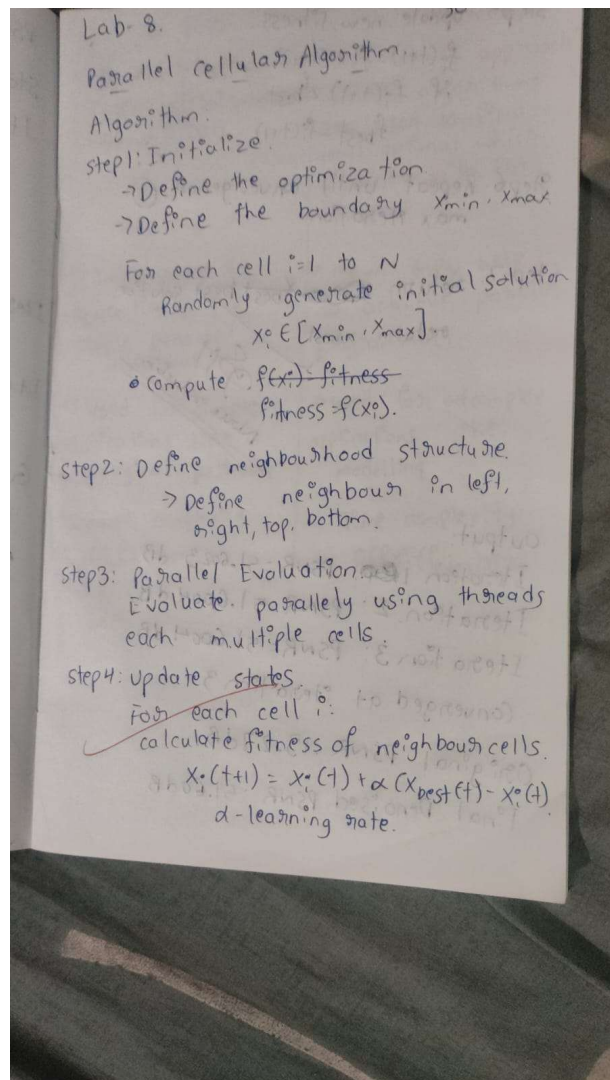


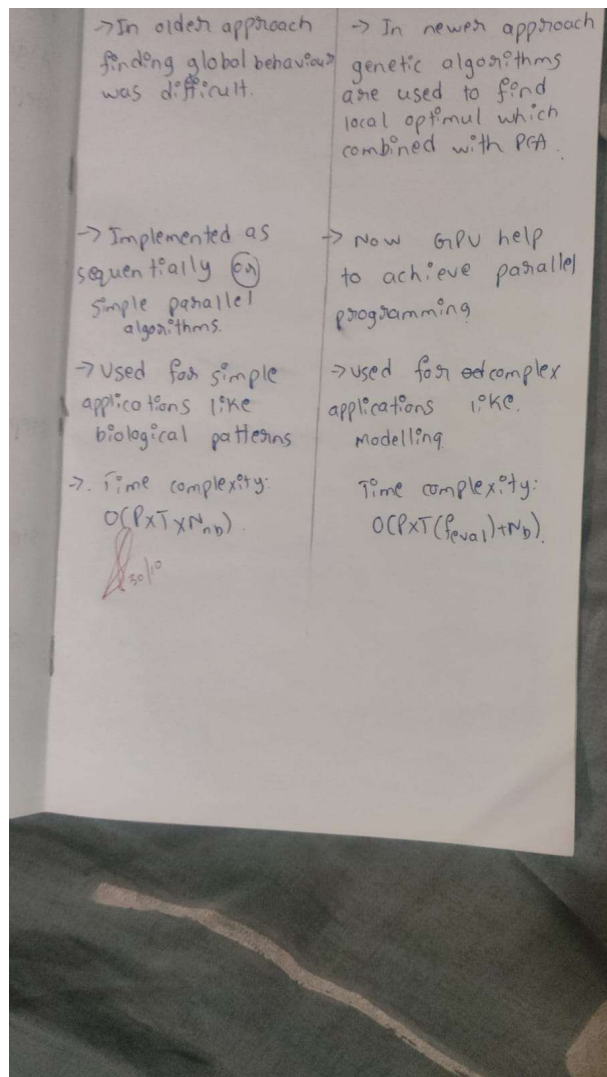
Program 7 :

Parallel Cellular Algorithm :

A parallel cellular algorithm is a computational model inspired by *cellular automata*, where a large problem is divided into many small units called cells, each performing simple computations. These cells update their states simultaneously based on predefined rules and the states of their neighboring cells. Because all cells operate in parallel, the algorithm is highly efficient for large-scale or grid-based problems such as image processing, simulation, pattern formation, and optimization. The parallelism enables faster computation and makes the algorithm scalable for execution on multi-core processors, GPUs, and distributed systems.

Algorithm :





Code :

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
```



```

def add_salt_and_pepper_noise(image, prob=0.05):
    """Adds salt-and-pepper noise to a grayscale image."""
    output = np.copy(image)
    total_pixels = image.size
    num_salt = np.ceil(prob * total_pixels / 2).astype(int)
    coords = [np.random.randint(0, i - 1, num_salt) for i in image.shape]
    output[coords[0], coords[1]] = 255
    num_pepper = np.ceil(prob * total_pixels / 2).astype(int)
    coords = [np.random.randint(0, i - 1, num_pepper) for i in image.shape]
    output[coords[0], coords[1]] = 0
    return output

def calculate_psnr(img1, img2):
    """Calculates Peak Signal-to-Noise Ratio (PSNR) in dB."""
    mse = np.mean((img1.astype(np.float64) - img2.astype(np.float64)) ** 2)
    if mse == 0:
        return 100.0
    max_pixel = 255.0
    psnr = 20 * np.log10(max_pixel / np.sqrt(mse))
    return psnr

def parallel_ca_denoise(noisy_image, original_img, max_iterations=10, convergence_threshold=0):
    """
    Applies 2D Cellular Automata to remove Salt-and-Pepper noise,
    tracking PSNR at each iteration.
    """
    current_image = noisy_image.copy()
    rows, cols = current_image.shape
    psnr_history = []
    for t in range(max_iterations):
        padded_image = np.pad(current_image, 1, mode='reflect')
        next_image = current_image.copy()
        changes_count = 0

        for r in range(rows):
            for c in range(cols):

                neighborhood = padded_image[r:r+3, c:c+3]
                center_pixel = neighborhood[1, 1]

                if center_pixel == 0 or center_pixel == 255:

                    neighbors_without_center = np.delete(neighborhood.flatten(), 4)

                    non_noise_neighbors = neighbors_without_center[
                        (neighbors_without_center > 0) & (neighbors_without_center < 255)

```

```

    ]

    new_state = center_pixel

    if non_noise_neighbors.size > 0:

        new_state = np.median(non_noise_neighbors).astype(np.uint8)
    else:

        new_state = np.median(neighbors_without_center).astype(np.uint8)

    if new_state != center_pixel:
        next_image[r, c] = new_state
        changes_count += 1

current_image = next_image.copy()

current_psnr = calculate_psnr(original_img, current_image)
psnr_history.append(current_psnr)

print(f'Iteration {t+1}: PSNR = {current_psnr:.4f} dB ({changes_count} pixels updated)')

if changes_count <= convergence_threshold:
    print(f'CA converged at iteration {t+1}.')
    break

return current_image, psnr_history

try:
    original_img = cv2.imread('lena.png', cv2.IMREAD_GRAYSCALE)
    if original_img is None:
        raise FileNotFoundError
except FileNotFoundError:
    print("Could not load 'lena.png'. Creating a synthetic image.")
    size = 200
    original_img = np.zeros((size, size), dtype=np.uint8)
    for i in range(size):
        original_img[i, :] = np.linspace(0, 255, size).astype(np.uint8)

noise_probability = 0.15
noisy_img = add_salt_and_pepper_noise(original_img, prob=noise_probability)

max_iters = 10

```

```

denoised_img, psnr_history = parallel_ca_denoise(noisy_img, original_img,
max_iterations=max_iters)

psnr_noisy = calculate_psnr(original_img, noisy_img)
psnr_denoised_final = psnr_history[-1] if psnr_history else calculate_psnr(original_img,
denoised_img)

print("\n--- Summary ---")
print(f'Original PSNR (Noise): {psnr_noisy:.2f} dB')
print(f'Final Denoised PSNR (CA): {psnr_denoised_final:.2f} dB')
print("1BM23CS246")

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(original_img, cmap='gray')
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].imshow(noisy_img, cmap='gray')
axes[1].set_title(f'Noisy Image ({noise_probability*100:.0f}% S&P, PSNR: {psnr_noisy:.2f} dB)')
axes[1].axis('off')

axes[2].imshow(denoised_img, cmap='gray')
axes[2].set_title(f'CA Denoised Image (PSNR: {psnr_denoised_final:.2f} dB)')
axes[2].axis('off')

plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 4))
plt.plot(range(1, len(psnr_history) + 1), psnr_history, marker='.', linestyle='-')
plt.title('PSNR Convergence for Parallel CA Denoising')
plt.xlabel('Iteration Number')
plt.ylabel('PSNR (dB)')
plt.grid(True)
plt.show()

```

Output :

Iteration 1: PSNR = 61.2913 dB (5980 pixels updated)

Iteration 2: PSNR = 61.2913 dB (0 pixels updated)

...

CA converged at iteration 2.

--- Summary ---

Original PSNR (Noise): 13.33 dB

Final Denoised PSNR (CA): 61.29 dB

1BM23CS246

Original Image

Noisy Image (15% S&P, PSNR: 13.33 dB)

CA Denoised Image (PSNR: 61.29 dB)

